

Energy-Efficient Heterogeneous Deployment of JPEG-AI on CPU-GPU-FPGA Platforms

George Paul*, Robert Nicol†, Boguslaw Obara*, and Deepayan Bhowmik*

*School of Computing, Newcastle University, Newcastle upon Tyne, UK

†STMicroelectronics, Edinburgh, UK

Abstract—JPEG-AI moves learned image compression toward standardised, interoperable bitstreams and is arguably considered to be one of the most efficient codecs. However, energy-efficient deployments are yet to be looked at, as they are challenging because highly parallel neural transforms coexist with serial, bitstream-critical components. Addressing this challenge, we present a profiling-informed heterogeneous implementation of JPEG-AI on a CPU-GPU-FPGA platform. The pipeline is split at the quantised latent, enabling a single packed-latent transfer per direction. In our heterogeneous deployment, we keep entropy coding/decoding and bitstream endpoints on the CPU; the reconstruction path is deployed on the GPU; and the INT8-suitable hyperprior subgraphs are offloaded to the FPGA resulting in mixed-precision offload being confined to entropy-parameter generation rather than pixel reconstruction. For a fair comparison, in addition to architecture-specific results, PCIe transfer, synchronisation, and host-orchestration overheads are measured explicitly, and bitrate is computed from stored bitstreams. Across 100 native-2K images and five target bitrates (0.12 – 1.00 bpp), our proposition resulted in the heterogeneous mapping matching GPU-only encoder runtime ($\pm 2.3\%$) while reducing encoder energy by 6.6%. For the decoder, runtime stays within 2.2% of GPU-only while energy decreases by 9.7%. Overall, end-to-end energy-delay product (EDP) improves by 9.7% relative to GPU-only and 17.0% relative to FPGA-only. We confirmed rate-distortion validations are intact with maximum GPU-only deviations of 0.002 bpp, 0.05 dB PSNR, or 0.0014 in MS-SSIM.

Index Terms—JPEG-AI, learned image compression, heterogeneous deployment, energy-efficient computing.

I. INTRODUCTION

Image compression is central to bandwidth- and energy-limited visual pipelines in satellite imaging, robotics, and autonomous vehicles [1], [2]. Traditional codecs such as JPEG and HEVC are widely used for their maturity and predictable complexity [3], [4]. Learned image compression (LIC) improves rate-distortion performance using neural analysis/synthesis transforms with learned entropy models, including hyperpriors and context modelling [5]. JPEG-AI brings LIC toward standardised, interoperable bitstreams [6].

Efficient JPEG-AI deployment is a systems problem rather than a single-kernel acceleration problem. The neural path contains residual attention units (ResAUs), strided convolutions, pixel-unshuffle/pixel-shuffle operators, quantisation, and hyperprior analysis/synthesis, while the entropy path contains

serial arithmetic coding and bitstream I/O. These components expose different parallelism, precision, memory-access, and control-flow characteristics. Transform layers favour GPU execution, compact hyperprior subgraphs can benefit from FPGA fixed-point dataflow, and entropy coding must remain deterministic, lossless over the transmitted symbols, and bitstream-correct making it a candidate for CPU.

Practical deployment is constrained by latency, throughput, and energy budgets [7]. Although modern systems are heterogeneous, learned codecs are often executed on a single device, leaving latency-energy trade-offs underexploited [8]. In JPEG-AI, the entropy interface and bitstream generation are normative, so heterogeneous execution must preserve deterministic rounding, tensor layout, symbol exchange, and the reconstruction path while accounting for PCIe transfer and host-orchestration overheads that can negate accelerator gains. This paper proposes a technique to efficiently deploy JPEG-AI compression on a heterogeneous platform, achieving FPGA-like energy efficiency while maintaining GPU-like runtime.

To the best of the authors' knowledge, this work is the first to realise JPEG-AI on a CPU-GPU-FPGA heterogeneous platform with unified runtime and energy analysis. We develop a profiling-driven, energy-aware deployment where stage-level runtime and energy measurements on CPU, GPU, and FPGA guide a codec-aware partition and device mapping that targets low energy-delay product (EDP) while preserving the entropy and bitstream endpoints. Only compact intermediates (quantised latents) cross devices, making transfer and orchestration costs explicit and repeatable. The main contributions are:

- **Energy-aware JPEG-AI profiling and partitioning** through stage-level runtime and energy characterisation that identifies bottlenecks and selects offload boundaries while preserving entropy and bitstream endpoints.
- **Heterogeneous deployment on CPU-GPU-FPGA** using a codec-aware mapping that exploits complementary device strengths while accounting for PCIe transfer and host-orchestration costs.
- **End-to-end deployment workflow** with ONNX [9] pipeline that exports, partitions, and executes pre-trained JPEG-AI subgraphs reproducibly across accelerators.

II. RELATED WORK

Learned image compression and standardisation: LIC uses neural analysis/synthesis transforms with learned entropy mod-

els: images are mapped to quantised latents that are entropy-coded under a learned distribution, then decoded back to pixels [10]. Hyperpriors and context models improve entropy estimation by transmitting side information and exploiting latent dependencies. JPEG-AI standardises learned codecs into interoperable bitstreams, increasing the importance of deployments that preserve the reference entropy interface and bitstream-correct behaviour.

Acceleration and heterogeneous deployment: Prior work accelerates learned codecs using GPUs, FPGAs, and ASICs to reduce latency or power [11]. FPGAs benefit from fixed-point streaming execution but require careful operator mapping and system integration [12]; ASICs offer higher efficiency at the cost of flexibility as models evolve [13]. Cross-device inference studies consider partitioning and scheduling under interconnect and orchestration overheads: overly fine splits increase PCIe traffic and synchronisation, while coarse mappings can underutilise accelerators [14]. Learned codecs further constrain valid cut points because probability modelling and entropy coding/decoding require consistent rounding, tensor layout, and symbol exchange to maintain bitstream correctness [15].

Our approach: We profile codec-aligned subgraphs on CPU, GPU, & FPGA over a bitrate sweep and use the measurements to choose a cut at the quantised latent $\hat{y}$, preserving the normative entropy/bitstream endpoints while making PCIe transfer and host orchestration costs explicit for low EDP.

III. METHODOLOGY

We target low latency and low energy for JPEG-AI on a CPU-GPU-FPGA platform by: (i) measuring codec-aligned subgraph runtime and energy across devices over a bitrate sweep, (ii) choosing a split that preserves the entropy and bitstream endpoints while minimising cross-device traffic, and (iii) executing the resulting schedule with explicit transfer and host-orchestration accounting.

A. Profiling

We benchmark JPEG-AI under three execution modes on the same image set and target rates. *CPU* executes the full codec on the host (Intel i7-12700K). *GPU* offloads neural transforms to CUDA on an RTX A2000 (FP16), while entropy coding/decoding and bitstream endpoints remain CPU-resident. *FPGA* uses a DPU subgraph-offload flow on an Alveo U50 (INT8 with int32 accumulators); any remaining layers execute on the CPU without changing the bitstream interface. In all modes the CPU is the normative bitstream endpoint; accelerators do not exchange tensors directly.

The measured workload is the complete codec path, not an isolated convolution benchmark. The analysis and synthesis transforms combine ResAU blocks, strided convolutions, residual additions, nonlinear activations, and pixel-unshuffle/pixel-shuffle resizing. The hyperprior path is smaller but still includes convolutional transforms and quantisation-dependent probability-parameter generation. The entropy path is scalar and serial compared with the neural transforms. This operator mix explains the relatively large per-image latency at

native 2K resolution and motivates a partition that keeps high-throughput tensor operators on the GPU, maps compact fixed-point-friendly subgraphs to the FPGA, and avoids moving entropy-state across devices.

Profiling follows the codec-aligned split in Fig. 1. Given $g_a(\cdot)$ and $g_s(\cdot)$ are the analysis and synthesis transforms, Q is the quantisation, $h_a(\cdot)$ and $h_s(\cdot)$ are hyperprior-analysis and hyperprior-synthesis transforms, S1a/S1b implement g_a+Q and emit the quantised latent $\hat{y}$; S2a/S4a implement h_a/h_s and produce entropy parameters; and S3a/S3b implement $g_s(\cdot)$. Entropy endpoints are treated as CPU-resident stages: arithmetic encoding (*AE*, the range coder in the encoder) and arithmetic decoding (*AD*, the range coder in the decoder), denoted S_{ent} .

Experiments process $N=100$ DIV2K validation images at native 2K resolution and five target bitrates: 0.12, 0.25, 0.50, 0.75, and 1.00 bpp. For each image and target rate, we run a complete encode producing a bitstream $\mathcal{B}$ and a complete decode producing the reconstructed image. Achieved bitrate is

$$\text{bpp} = \frac{|\mathcal{B}|}{HW}, \quad (1)$$

where $|\mathcal{B}|$ is the bitstream length, computed as $8 \times$ the stored byte length, and H and W are image height and width. To isolate decoder measurements from encoder variability, decode profiling reuses identical stored bitstreams across modes.

Runtime is measured with CPU wall-clock timers, CUDA events (GPU), and FPGA runtime counters. Stage time includes required synchronisation and transfers for that mode (including pack/unpack and PCIe DMA for $\hat{y}$). Table I reports per-stage means and end-to-end totals. It shows that g_a+Q and g_s dominate at 2K and benefit most from GPU execution, whereas the hyperprior stages are comparatively small and map efficiently to the FPGA offload path, motivating the split in Section III-B.

B. Codec partitioning

JPEG-AI is a learned transform codec trained to minimise a Lagrangian rate-distortion objective:

$$\min_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[D(\mathbf{x}, \hat{\mathbf{x}}) + \lambda R \right], \quad (2)$$

where θ are model parameters, $\mathbf{x}$ is input image, $\hat{\mathbf{x}}$ is the reconstruction, $\mathcal{D}$ is the data distribution, $D(\cdot)$ is a distortion measure, R is the expected rate in bits, and λ selects a bitrate.

Given $\mathbf{x} \in \mathbb{R}^{1 \times 3 \times H \times W}$, the analysis transform produces

$$\mathbf{y} = g_a(\mathbf{x}), \quad \hat{\mathbf{y}} = Q(\mathbf{y}) \in \mathbb{Z}^{1 \times C \times (H/\hat{s}) \times (W/\hat{s})}, \quad (3)$$

where C is the latent channel count, and $\hat{s}$ is the spatial downsampling factor of g_a . Quantisation uses a fixed rounding rule; with step size Δ ,

$$Q(z) = \text{round}_{\text{tie-to-even}} \left(\frac{z}{\Delta} \right). \quad (4)$$

A hyperprior generates side information and entropy parameters

$$\mathbf{z} = h_a(\hat{\mathbf{y}}), \quad \hat{\mathbf{z}} = Q(\mathbf{z}), \quad \boldsymbol{\psi} = h_s(\hat{\mathbf{z}}), \quad (5)$$

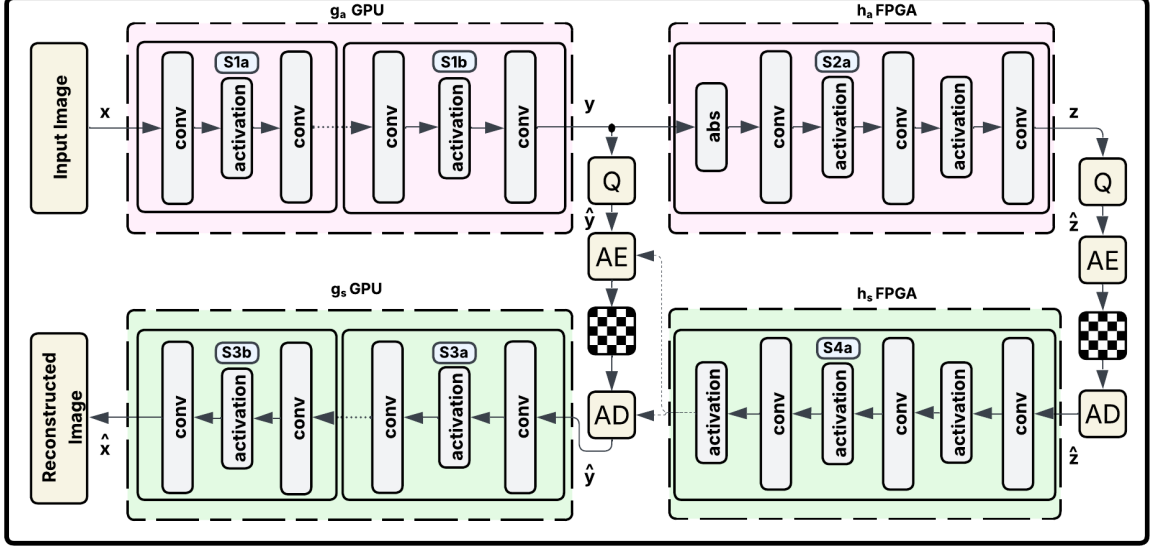


Fig. 1. JPEG-AI heterogeneous partitioning overview. The split is placed at the quantised latent $\hat{y}$ so that only packed $\hat{y}$ crosses PCIe once per direction, with Direct Memory Access (DMA) transfers overlapped with compute where possible. The bandwidth-dominated analysis/synthesis transforms (g_a, g_s) execute on the GPU, the hyperprior transforms (h_a, h_s) execute on the FPGA via INT8 DPU subgraph offload, and arithmetic encoding/decoding (for both $\hat{y}$ and $\hat{z}$) remains CPU-resident to preserve the entropy and bitstream interfaces. Codec-aligned subgraphs used for profiling and deployment are: S1a (early layers of g_a), S1b (remaining g_a layers plus quantiser Q producing $\hat{y}$), S2a/S4a (h_a/h_s), and S3a/S3b (early and late layers of g_s up to reconstruction).

where ψ conditions the entropy model $p(\hat{y} | \psi)$. The bitstream $\mathcal{B}$ is formed by entropy coding $\hat{z}$ and $\hat{y}$; under ideal range coding,

$$|\mathcal{B}| \approx \sum_i -\log_2 p(\hat{z}_i) + \sum_j -\log_2 p(\hat{y}_j | \psi), \quad (6)$$

where i and j index elements of $\hat{z}$ and $\hat{y}$, respectively.

We partition at the discrete latent $\hat{y}$ in Eq. (3). Three split classes were considered. Earlier feature-map cuts expose more parallel work but move large intermediate tensors across PCIe. Later cuts reduce tensor movement but entangle probability-model state, symbol order, and entropy-parameter generation with the device boundary. The $\hat{y}$ cut is therefore a compact deterministic interface: it is after transform quantisation, before CPU entropy coding, and independent of the arithmetic coder state. Table I shows S1a/S1b and S3a/S3b dominate runtime at 2K and favour GPU execution, while S2a/S4a are compact and favourable for FPGA execution via DPU offload. We therefore map g_a/g_s to GPU, h_a/h_s to FPGA, and entropy/bitstream endpoints to CPU.

C. Heterogeneous deployment

We export the encoder/decoder to ONNX [9] subgraphs (S1a/S1b, S2a/S4a, S3a/S3b). As shown in Table II, the execution maps g_a/g_s to GPU, h_a/h_s to FPGA via DPU subgraph offload, and AE/AD plus bitstream handling to CPU. Only the packed $\hat{y}$ is transferred once per direction via PCIe; $\mathcal{B}$ is formed and consumed on the CPU.

The interface at $\hat{y}$ fixes tensor layout, static shapes/strides (padding to multiples of $\hat{s}$), symbol packing endianness,

and ties-to-even rounding per Eq. (4), enabling bit-exact PACK/UNPACK across devices. This split also protects compression/decompression performance. The reconstructed image is $\hat{x} = g_s(\hat{y})$; $g_a, \hat{y}$, and g_s remain on the GPU path, so mixed-precision FPGA execution does not alter the pixel reconstruction transform. FPGA DPU offload is restricted to h_a/h_s , which generates entropy parameters ψ for lossless arithmetic coding of already-quantised symbols. Any small entropy-model quantisation effect is therefore reflected in the measured stored-bitstream bpp rather than hidden as a change in distortion. GPU transforms execute in FP16; FPGA DPU subgraphs execute in INT8 with int32 accumulators.

We use pinned host buffers, double buffering, and asynchronous DMA to overlap transfer and compute. For steady-state streaming:

$$T_{\text{enc}} \approx \max\{T_{S1a} + T_{S1b}, T_{\text{dma}} + T_{S2a} + T_{\text{ent,enc}}\}, \quad (7)$$

$$T_{\text{dec}} \approx \max\{T_{S3a} + T_{S3b}, T_{\text{ent,dec}} + T_{\text{dma}} + T_{S4a}\}, \quad (8)$$

where T_{S*} are per-image subgraph times, T_{dma} includes pack/unpack plus PCIe DMA of $\hat{y}$, and $T_{\text{ent,*}}$ are CPU entropy times.

For each image, transfer timing is bracketed around host packing, host-to-device/device-to-host DMA submission, FPGA/GPU synchronisation, and unpacking. Energy is integrated over the same per-image execution window:

$$E_{i,p} = \sum_{d \in \{\text{CPU}, \text{GPU}, \text{FPGA}\}} \int_{t_{0,i,p}}^{t_{1,i,p}} (P_d(t) - P_d^{\text{idle}}) dt, \quad (9)$$

TABLE I
PER-STAGE RUNTIME (S/IMG) AT 2K FOR CPU, GPU, AND FPGA MODES
(MEANS OVER $N=100$). **Bold** = FASTEST.

Target bpp	Stage	CPU	GPU	FPGA
0.12	S1a/S1b (g_a+Q)	118.4	26.9	41.6
	S2a (h_a)	12.9	3.9	2.6
	S_{ent} (enc, AE)	8.9	8.6	8.6
	S_{ent} (dec, AD)	7.6	7.2	7.3
	S4a (h_s)	6.9	2.9	2.1
	S3a/S3b (g_s)	20.8	9.6	12.8
E2E total (enc+dec)		175.5	59.1	75.0
0.25	S1a/S1b (g_a+Q)	112.7	25.4	39.4
	S2a (h_a)	12.1	3.7	2.5
	S_{ent} (enc, AE)	8.4	8.1	8.1
	S_{ent} (dec, AD)	7.3	6.9	6.9
	S4a (h_s)	6.6	2.8	2.0
	S3a/S3b (g_s)	19.6	9.1	12.2
E2E total (enc+dec)		166.7	56.0	71.1
0.50	S1a/S1b (g_a+Q)	114.9	26.1	40.2
	S2a (h_a)	12.5	3.8	2.6
	S_{ent} (enc, AE)	8.7	8.4	8.4
	S_{ent} (dec, AD)	7.5	7.1	7.1
	S4a (h_s)	6.8	2.9	2.1
	S3a/S3b (g_s)	20.2	9.4	12.6
E2E total (enc+dec)		170.6	57.7	73.0
0.75	S1a/S1b (g_a+Q)	123.6	27.6	42.8
	S2a (h_a)	13.4	4.0	2.7
	S_{ent} (enc, AE)	9.2	8.9	8.9
	S_{ent} (dec, AD)	7.8	7.4	7.5
	S4a (h_s)	7.2	3.0	2.2
	S3a/S3b (g_s)	21.6	10.0	13.4
E2E total (enc+dec)		182.8	60.9	77.5
1.00	S1a/S1b (g_a+Q)	109.8	24.6	37.6
	S2a (h_a)	11.8	3.6	2.4
	S_{ent} (enc, AE)	8.1	7.9	7.9
	S_{ent} (dec, AD)	7.1	6.7	6.7
	S4a (h_s)	6.4	2.7	1.9
	S3a/S3b (g_s)	18.9	8.7	11.7
E2E total (enc+dec)		162.1	54.2	68.2

where $p \in \{\text{enc}, \text{dec}\}$, $P_d(t)$ is the sampled device power, and P_d^{idle} is the pre-measured idle level for that device. Thus pack/unpack, PCIe DMA, synchronisation, CPU orchestration, and accelerator execution are all included in the measured runtime and active energy.

IV. EXPERIMENTAL RESULTS

A. Setup and metrics

Experiments run on an Intel i7-12700K (CPU), NVIDIA RTX A2000 (GPU), and Xilinx Alveo U50 (FPGA). We considered commonly used JPEG AI compression test conditions and evaluated the proposed hardware deployment on $N=100$ DIV2K validation images [16] at native 2K resolution, sweeping five target bitrates: 0.12, 0.25, 0.50, 0.75, and 1.00bpp. We report four execution modes: *CPU* (full codec on host), *GPU* (GPU executes g_a/g_s in FP16; entropy coding/decoding and bitstream I/O on CPU), *FPGA* (INT8

TABLE II
SUBGRAPHS AND DEVICE PLACEMENT AROUND $\hat{y}$.

Subgraph	Role	HW	Justification
S1a	Early encoder (down/conv)	GPU (FP16)	Large activations; bandwidth/throughput dominated at 2K.
S1b	Late encoder + quantise	GPU (FP16)	Emits $\hat{y}$; keeps the split compact.
S2a	Hyper-analysis	FPGA (INT8)	Compact convs; efficient DPU dataflow.
S4a	Hyper-synthesis	FPGA (INT8)	Produces ψ locally; avoids shipping state.
S3a	Early decoder (up/conv)	GPU (FP16)	Upsampling/conv benefits from GPU parallelism.
S3b	Late decoder (reconstruct)	GPU (FP16)	Keeps reconstruction local; avoids mid-pipeline device-switching
S_{ent}	AE/AD + bitstream I/O	CPU	Normative entropy endpoint and bitstream handling.

DPU codec subgraph offload; CPU remains the bitstream endpoint), and the proposed *heterogeneous* mapping that cuts at the quantised latent $\hat{y}$ and transfers a single packed- $\hat{y}$ tensor per direction (Section III). Fixed scheduling and bounded PCIe transfers are used to reduce run-to-run variance.

The reported runtime is for the complete image-codec pipeline. It includes ResAU-heavy analysis/synthesis transforms, pixel-unshuffle/pixel-shuffle resizing, quantisation, hyperprior inference, arithmetic coding/decoding, pack/unpack, PCIe DMA, synchronisation, and host orchestration. The study therefore compares hardware mappings under an identical codec workload rather than reporting isolated accelerator-kernel throughput.

For image $i=1, \dots, N$, target rate r , and path $p \in \{\text{enc}, \text{dec}\}$, mean runtime ($\bar{t}_p$) and energy ($\bar{E}_p$) are

$$\bar{t}_p(r) = \frac{1}{N} \sum_{i=1}^N t_{p,i}(r), \quad \bar{E}_p(r) = \frac{1}{N} \sum_{i=1}^N E_{p,i}(r), \quad (10)$$

with 95% bootstrap confidence intervals (10^4 resamples). The Energy-Delay Product (EDP), measured in (kJ $\cdot$ s/img), is expressed below:

$$\text{EDP}_p(r) = \bar{E}_p(r) \bar{t}_p(r). \quad (11)$$

End-to-end values use $\bar{t}_{\text{e2e}} = \bar{t}_{\text{enc}} + \bar{t}_{\text{dec}}$, $\bar{E}_{\text{e2e}} = \bar{E}_{\text{enc}} + \bar{E}_{\text{dec}}$, and $\text{EDP}_{\text{e2e}} = \bar{E}_{\text{e2e}} \bar{t}_{\text{e2e}}$.

Measurements and codec checks: Runtime uses CPU wall-clock timers, CUDA events, and FPGA runtime counters. Power is sampled via RAPL [17], `nvidia-smi` [18], and `xbutil` [19]; energy is obtained by per-image integration with idle power removed and summed across devices as in Eq. (9). Each mode runs a full encode/decode path from stored bitstreams and computes bpp from the actual byte length. The proposed mapping does not bypass entropy coding, change the trained rate-distortion objective, or move the synthesis

TABLE III
RATE-DISTORTION VALIDATION AT NATIVE 2K RESOLUTION; $N=100$ IMAGES. EACH PLATFORM REPORTS BITRATE (BPP), PSNR (DB), AND MS-SSIM.

Target bpp	CPU			GPU			FPGA			Heterogeneous		
	bpp	PSNR	MS-SSIM	bpp	PSNR	MS-SSIM	bpp	PSNR	MS-SSIM	bpp	PSNR	MS-SSIM
0.12	0.120	28.51	0.9359	0.120	28.50	0.9357	0.122	28.50	0.9343	0.121	28.52	0.9350
0.25	0.250	31.05	0.9566	0.250	31.06	0.9567	0.252	31.03	0.9560	0.251	31.07	0.9562
0.50	0.500	33.82	0.9739	0.500	33.84	0.9731	0.501	33.77	0.9723	0.501	33.81	0.9735
0.75	0.749	35.52	0.9788	0.749	35.54	0.9784	0.750	35.47	0.9795	0.751	35.51	0.9798
1.00	1.000	36.93	0.9844	1.000	36.93	0.9840	1.001	36.81	0.9834	1.001	36.88	0.9849

transform to the FPGA. Since reconstruction is determined by the transmitted $\hat{y}$ and the unchanged g_s path, mixed-precision hyperprior offload can affect coding cost only through the probability model used to encode symbols; this effect is included in the measured bpp. The reported gains therefore come from deployment and device mapping rather than relaxed codec performance.

Rate-distortion validation: Table III reports bitrate, PSNR, and MS-SSIM for CPU, GPU, FPGA, and heterogeneous execution. The values are closely matched across all operating points, indicating that the proposed heterogeneous deployment preserves JPEG-AI codec behaviour. In particular, the heterogeneous path differs from GPU-only by at most 0.002 bpp, 0.05 dB PSNR, and 0.0014 in MS-SSIM, confirming that confining FPGA mixed-precision execution to hyperprior entropy-parameter generation does not materially degrade compression or decompression performance. The small differences observed for the FPGA and heterogeneous modes are consistent with probability-model and implementation-level numerical effects rather than a changed reconstruction path.

B. Heterogeneous performance and trade-offs

Table IV summarises heterogeneous encoder/decoder runtime, energy, and EDP across bitrates. On average, encoding dominates the end-to-end cost (37.5 s/img and 8.58 kJ/img) relative to decoding (18.9 s/img and 4.17 kJ/img), accounting for $\approx 66\%$ of end-to-end runtime and $\approx 67\%$ of end-to-end energy. Across bitrates, heterogeneous end-to-end totals span 53.0-59.6 s/img and 11.50-14.45 kJ/img.

Runtime: Fig. 2 plots end-to-end runtime versus target bitrate. The heterogeneous design remains close to GPU-only because the bandwidth-dominated transforms (g_a/g_s) stay GPU-resident and only the compact $\hat{y}$ crosses the interconnect, making transfer and orchestration overheads limited.

Energy: Fig. 3 shows end-to-end energy versus target bitrate. Offloading the hyperprior (h_a/h_s) to the FPGA reduces energy relative to GPU-only across bitrates, while retaining GPU throughput for the dominant transforms.

Energy-delay product: Fig. 4 reports end-to-end EDP, capturing the combined effect of GPU-class latency with reduced energy from FPGA hyperprior execution. Across the measured

TABLE IV
HETEROGENEOUS JPEG-AI RESULTS AT NATIVE 2K RESOLUTION; $N=100$ IMAGES. VALUES ARE PER-PATH RUNTIME IN S/IMG, ENERGY IN KJ/IMG, AND EDP IN KJ·S/IMG.

bpp	Enc. t (s)	Enc. E (kJ)	Enc. EDP (kJ·s)	Dec. t (s)	Dec. E (kJ)	Dec. EDP (kJ·s)
0.12	38.5	9.03	347.5	19.3	4.34	83.7
0.25	36.4	8.01	291.5	18.4	3.84	70.7
0.50	37.5	8.58	321.9	19.0	4.01	76.2
0.75	39.6	9.75	386.2	20.0	4.70	94.1
1.00	35.3	7.54	266.2	17.7	3.96	70.1
Avg.	37.5	8.58	322.7	18.9	4.17	79.0

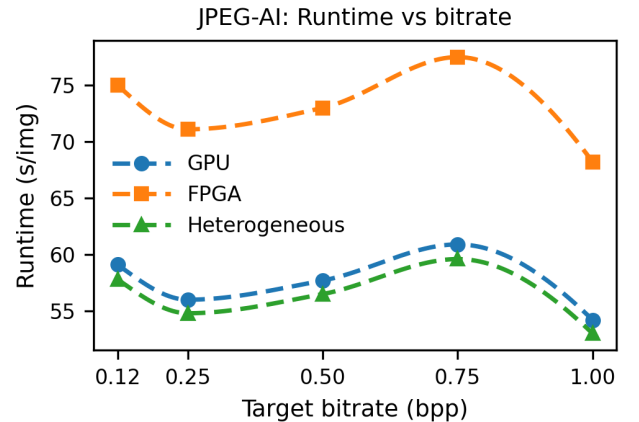


Fig. 2. End-to-end runtime (enc+dec, s/img) versus target bitrate (bpp) for GPU, FPGA, and heterogeneous execution; lower is better.

bitrates, the heterogeneous mapping yields the lowest EDP among the three accelerator configurations.

V. CONCLUSION

We present a energy-efficient heterogeneous deployment of JPEG-AI that partitions at the quantised latent $\hat{y}$ to bound PCIe traffic while keeping entropy coding/decoding and bitstream endpoints CPU-resident. The GPU executes the bandwidth-dominated ResAU and pixel-shuffle/unshuffle

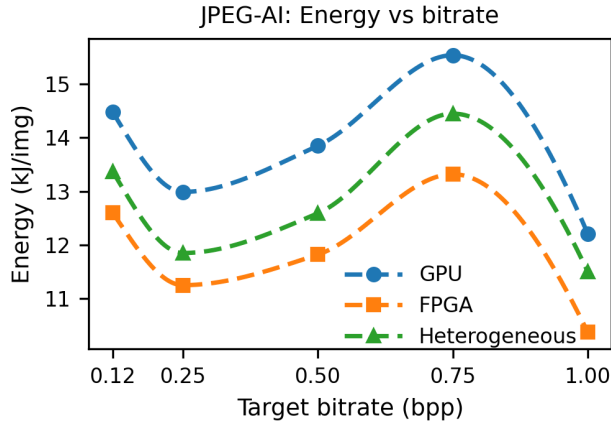


Fig. 3. End-to-end energy (enc+dec, kJ/img) versus target bitrate (bpp) for GPU, FPGA, and heterogeneous execution; lower is better.

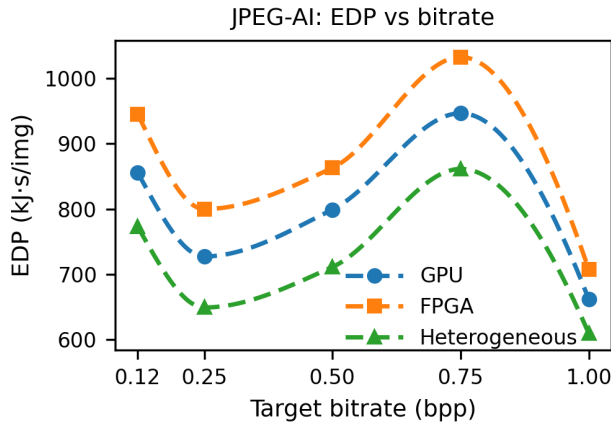


Fig. 4. End-to-end EDP (enc+dec, kJ-s/img) versus target bitrate (bpp) for GPU, FPGA, and heterogeneous execution; lower is better.

analysis/synthesis transforms g_a/g_s , the FPGA accelerates the compact hyperprior h_a/h_s , and the CPU performs arithmetic coding/decoding and orchestration. This confines fixed-point offload to entropy-parameter generation while preserving the transmitted latent interface and GPU reconstruction path. Rate-distortion validation shows maximum GPU-only deviations of 0.002 bpp, 0.05 dB PSNR, and 0.0014 in MS-SSIM for the heterogeneous path, supporting codec-behaviour preservation. Across $N=100$ native-2K DIV2K images and five target bitrates, the heterogeneous mapping matches GPU-only encoder runtime within $\approx 2.3\%$ while reducing encoder energy by $\approx 6.6\%$. For the decoder, runtime stays within $\approx 2.2\%$ of GPU-only while energy decreases by $\approx 9.7\%$. Overall, end-to-end EDP improves by $\approx 9.7\%$ versus GPU-only and $\approx 17.0\%$ versus FPGA-only, yielding a better latency-energy trade-off than either single-accelerator baseline.

REFERENCES

- [1] C. Schwartz, I. Sander, F. Bruhn, M. Persson, J. Ekblad, and C. Fuglesang, "Satellite image compression guided by regions of interest," *Sensors*, vol. 23, no. 2, 2023. [Online]. Available: <https://www.mdpi.com/1424-8220/23/2/730>
- [2] P. H. Chan, G. Souvalioti, A. Huggett, G. Kirsch, and V. Donzella, "The data conundrum: Compression of automotive imaging data and deep neural network based perception," in *Proceedings Society for Imaging Science and Technology London Imaging Meeting 2021*, vol. 2021, no. 1. LIM, 2021, pp. 78–82.
- [3] G. K. Wallace, "The JPEG still picture compression standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992.
- [4] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012.
- [5] J. Ballé, D. C. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," *arXiv*, vol. abs/1802.01436, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:3611540>
- [6] E. Alshina, J. Ascenso, and T. Ebrahimi, "JPEG AI: The first international standard for image coding based on an end-to-end learning-based approach," *IEEE MultiMedia*, vol. 31, no. 4, pp. 60–69, 2024.
- [7] X. Wang, Z. Tang, J. Guo, T. Meng, C. Wang, T. Wang, and W. Jia, "Empowering edge intelligence: A comprehensive survey on on-device AI models," *ACM Comput. Surv.*, vol. 57, no. 9, Apr. 2025. [Online]. Available: <https://doi.org/10.1145/3724420>
- [8] T. Ali, D. Bhowmik, and R. Nicol, "Energy aware computer vision algorithm deployment on heterogeneous architectures," *Discover Electronics*, vol. 2, no. 1, p. 42, 2025.
- [9] ONNX Contributors, "ONNX: Open neural network exchange," <https://onnx.ai>, 2019.
- [10] C.-H. Huang and J.-L. Wu, "Unveiling the future of human and machine coding: A survey of end-to-end learned image compression," *Entropy*, vol. 26, no. 5, 2024. [Online]. Available: <https://www.mdpi.com/1099-4300/26/5/357>
- [11] C. Chen, H. Zhang, K. Guo, X. Yu, W. Qiu, Z. Qi, and H. Guan, "Exploring efficient hardware accelerator for learning-based image compression," *Trans. Comput.-Aided Des. Integr. Cir. Sys.*, vol. 44, no. 6, pp. 2204–2217, Jun. 2025. [Online]. Available: <https://doi.org/10.1109/TCAD.2024.3515856>
- [12] H. Sun, J. Wang, S. Liu, S. Kimura, and M. Fujita, *Learned Image Codec on FPGA: Algorithm, Architecture and System Design*. New York, NY, USA: Association for Computing Machinery, 2025, p. 328–331. [Online]. Available: <https://doi.org/10.1145/3658617.3698489>
- [13] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Commun. ACM*, vol. 63, no. 7, pp. 48–57, Jun. 2020. [Online]. Available: <https://doi.org/10.1145/3361682>
- [14] T. Ali, G. Paul, R. Nicol, and D. Bhowmik, "Scheduling algorithms on heterogeneous architecture for efficient vision systems," in *Proceedings of SPIE Applications of Digital Image Processing XLVII*, A. G. Tescher and T. Ebrahimi, Eds., vol. 13137, International Society for Optics and Photonics (SPIE). SPIE, 2024, p. 131370F. [Online]. Available: <https://doi.org/10.1117/12.3031278>
- [15] J. Yu, S. Mai, P. Zhang, Y. Jiang, and J. Cheng, "Activation and weight distribution balancing for optimal post-training quantization in learned image compression," in *Proceedings of the 33rd ACM International Conference on Multimedia*, ser. MM '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 7959–7967. [Online]. Available: <https://doi.org/10.1145/3746027.3755218>
- [16] E. Agustsson and R. Timofte, "NTIRE 2017 challenge on single image super-resolution: Dataset and study," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, Jul. 2017.
- [17] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le, "RAPL: Memory power estimation and capping," in *Proceedings of the 16th ACM/IEEE International Symposium on Low Power Electronics and Design*, ser. ISLPED '10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 189–194. [Online]. Available: <https://doi.org/10.1145/1840845.1840883>
- [18] NVIDIA Corporation, "NVIDIA system management interface (nvidia-smi)," <https://developer.nvidia.com/nvidia-system-management-interface>, 2024.
- [19] A. Xilinx, "Xilinx board utility (xbutil)," <https://docs.amd.com/r/en-US/ug1301-xrt/xbutil-Utility>, 2024.